CS450: Artificial Intelligence
Minimax Konane

(110 points – 100 program, 10 pts evaluation writeup)

INTRODUCTION

For this lab you will implement the minimax adversarial search algorithm with alpha-beta pruning on the game of Konane, also known as Hawaiian Checkers. The game is played on an N by N board of black and white pieces. N may be as small as 4 or as large as 8. The board shown below is 8 by 8, and uses 'B' for black pieces and 'W' for white pieces.

```
   0 1 2 3 4 5 6 7
0  B W B W B W B W
1  W B W B W B W B
2  B W B W B W B W
3  W B W B W B W B
4  B W B W B W B W
5  W B W B W B W B
6  B W B W B W B W
7  W B W B W B W B
```

In order to play Konane, the black player goes first and removes one black piece from the corner or the center of the board. For an 8 by 8 board, this equates to positions (0,0), (3,3), (4,4), or (7,7). Next the white player removes a white piece adjacent to the space created by the first move. Then the players alternate moves, each jumping one of their own pieces over one horizontally or vertically adjacent opponent's piece, landing in an empty space on the other side, and removing the jumped piece. If desired, this may be continued in a multiple jump, as long as the same piece is moved in a straight line. The game ends when one player can no longer move and that player is considered the loser. Here's another description: http://math.berkeley.edu/~berlek/cgt/konane.html

I will provide you with a Python implementation of the Konane game. You can find the Python class on my website. You will implement minimax with alpha-beta pruning and then focus on finding a good evaluation function for estimating the utility of a given Konane board. As mentioned in class, "it should be clear that the performance of a game-playing program is dependent on the quality of its evaluation function" (Russell and Norvig, page 171).

In order to become familiar with python, you should review the tutorials provided on my website – plus, I'm sure you will find many on your own. Python is becoming more and more popular, and it's a good language with which to have some experience. The code I'm distributing will only work under Python 3 (Python 3 is not backward compatible).

There are also some online resources on how to play konane – youtube videos and online implementations. Seek those out to better understand the game.

You may work with a partner on this lab. If you work as a team, turn in only one copy of the lab and be sure that both of your names appear at the top of every file.

LAB INSTRUCTIONS (and notes)

1. Download konane.py from the AutoLab "handout" for the Konane lab.
2. Save konane.py to your own directory. Read through the class definitions.
3. Try executing this file by doing: `python konane.py`
   This will play one game between a `SimplePlayer` and a `RandomPlayer` on a 4 by 4 board.
4. Rather than playing one game at a time, you can play a series of games using the `playNGames` method. Try playing 10 games between the `SimplePlayer` and the `RandomPlayer` on a 8 by 8 board by changing the last two lines in the file to:

```
game = Konane(8)
game.playNGames(10, SimplePlayer(8), RandomPlayer(8), 0)
```

   Notice that the last argument determines whether the moves and the board will be displayed. When this argument is 0, only the final outcome will be shown.

5. Try playing a 6 by 6 game against the `RandomPlayer` by changing the last two lines in the file to:

```
game = Konane(6)
game.playOneGame(HumanPlayer(), RandomPlayer(6), 1)
```

6. Notice that the `Player` class is the base class for creating a Konane player. Implement a `MinimaxPlayer` class that inherits from both the `Player` class and the `Konane` class as shown below.

```
class MinimaxPlayer(Konane, Player):

    def __init__(self, size, depthLimit):
        Konane.__init__(self, size)
        self.limit = depthLimit

    def initialize(self, side):
        #complete this

    def getMove(self, board):
        #complete this

    def eval(self, board):
        #complete this - this will be your evaluation function. High
values should be good for max.
```

Notice that the first line of the `MinimaxPlayer` constructor calls the constructor of the `Konane` base class. Also notice that the depth limit for the search is passed in as an argument. You should be able to use your `MinimaxPlayer` in the following way:

```
game = Konane(8)
game.playNGames(2, MinimaxPlayer(8, 2), MinimaxPlayer(8, 1), 0)
```

Executing the above commands should demonstrate that the minimax player which searches to depth 2 will outperform a minimax player which only searches to depth 1. Your eval function may not "look ahead" and investigate resulting boards. The depth should be controlled by the depthLimit.

```
Game 0
MinimaxDepth2 wins
Game 1
MinimaxDepth2 wins
MinimaxDepth2 2 MinimaxDepth1 0
```

You should not make any changes to the `Konane` class. However, because the `MinimaxPlayer` class inherits from the `Konane` class, you are welcome to override any of the methods provided in the `Konane` class.

7. Set the `self.name` class variable in MinimaxPlayer to a unique team name for tournament play.
8. Important details for getting your implementation to agree with autolab's implementation (and the correct count of number of nodes pruned)... You should have two base cases for when your minimax algorithms return rather than recursing:
    a. You've reached the max depth
    b. There are no more moves. In this case, do NOT call eval. Just return float("inf") or -float("inf") to indicate a win/loss. If you call eval in this situation, you will have issues with autolab telling you that you're evaluating extra boards.

9. I strongly advise getting the basic minimax code working (you will receive half-credit on autolab for this) and then working on the pruning.
10. The complicating factor between implementing the alpha-beta pruning algorithm from the pseudocode and the konane alpha-beta is the need to find the move that is associated with the best evaluation. To do this easily, handle the top level of the alpha-beta process separately in getMove and start the alpha-beta pruning process with a call to the min layer for each possible top-level move.

TOURNAMENT PLAY

Each Konane player should conduct a minimax search to depth 4. The key difference between players will be in how they evaluate boards. Given two Konane players with fixed static evaluators, the game outcome will be deterministic. Therefore, each Konane player will play every other Konane player in the tournament twice, one time going first and the other time going second. Each player will receive a +1 for a win and a -1 for a loss. The winner of the tournament will be determined by the cumulative score over all of these games. It will be possible to have multiple winners.

We will hold the tournament to determine whose program is the ultimate Konane player. Extra credit will be awarded for placing first, second or third.

WHAT TO HAND IN

- Do **not** turn in the class definitions provided in the file `konane.py`. Instead turn in a file containing only the class definition for your minimax player. Submit this as `konane`.
    1. The .py file that you submit should contain only your class (none of the base classes). It doesn't matter what the actual file name of the file you submit is.
    2. It should have an "from konane import *" line at the top of the file.
    3. Your class should be named MinimaxPlayer
- Also turn in an ascii text file describing your evaluation function. Discuss the various features you considered including as part of the evaluation function. How did you determine which features were the most effective? Submit this as `Konane Evaluation`.

If you worked as a team be sure to include both partner's names at the top of each file.

Thanks
Many thanks to Lisa Meeden for allowing me to use this assignment!